

LAPORAN TUGAS 5
PRAKTIKUM SINKRONISASI SISTEM OPERASI



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Mohamad Qowi Aizani (2410651003)

PRODI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH
JEMBER 2025

Sistem Booking Hotel (akhiran NIM ganjil)

1. Source code

```
// hotel_booking.c
// Sistem Booking Hotel (10 customer, 5 kamar)
// - Semaphore untuk mengatur jumlah kamar tersedia
// - Mutex untuk melindungi operasi print dan alokasi room
// - Customer mencoba booking (butuh random 1-3s), stay 5s, max 3 percobaan
// - Jika gagal akan menunggu 2s lalu retry
//
// Compile: gcc hotel_booking.c -o hotel_booking -pthread
// Run: ./hotel_booking

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#include <time.h>
#include <errno.h>
#include <stdarg.h>

#define TOTAL_ROOMS 5
#define TOTAL_CUSTOMERS 10
#define MAX_TRIES 3
#define STAY_TIME 5    // detik menginap
#define RETRY_WAIT 2   // detik menunggu sebelum coba lagi

sem_t rooms_sem;        // mengatur jumlah kamar tersedia
int rooms[TOTAL_ROOMS]; // 0 = kosong, 1 = terisi
pthread_mutex_t rooms_mutex; // melindungi array rooms
pthread_mutex_t print_mutex; // melindungi output agar rapi
```

```
// Helper: ambil timestamp HH:MM:SS
```

```
void now_str(char *buf, size_t sz) {  
    time_t t = time(NULL);  
    struct tm *tm = localtime(&t);  
    strftime(buf, sz, "%H:%M:%S", tm);  
}
```

```
// Print ter-sinkron (menggunakan print_mutex)
```

```
void sync_print(const char *format, ...) {  
    va_list ap;  
    char timestr[16];  
    now_str(timestr, sizeof(timestr));  
  
    pthread_mutex_lock(&print_mutex);  
    printf("[%s] ", timestr);  
  
    va_start(ap, format);  
    vprintf(format, ap);  
    va_end(ap);  
  
    fflush(stdout);  
    pthread_mutex_unlock(&print_mutex);  
}
```

```
// Assign first available room, returns room index (0..TOTAL_ROOMS-1) or -1 jika  
tidak ada
```

```
int assign_room(int customer_id) {  
    int assigned = -1;  
    pthread_mutex_lock(&rooms_mutex);  
    for (int i = 0; i < TOTAL_ROOMS; ++i) {
```

```

        if (rooms[i] == 0) {
            rooms[i] = 1;
            assigned = i;
            break;
        }
    }
    pthread_mutex_unlock(&rooms_mutex);
    return assigned;
}

// Release room by index
void release_room(int room_index) {
    pthread_mutex_lock(&rooms_mutex);
    rooms[room_index] = 0;
    pthread_mutex_unlock(&rooms_mutex);
}

// Count available rooms (for printing)
int available_rooms_count() {
    int cnt = 0;
    pthread_mutex_lock(&rooms_mutex);
    for (int i = 0; i < TOTAL_ROOMS; ++i) if (rooms[i] == 0) cnt++;
    pthread_mutex_unlock(&rooms_mutex);
    return cnt;
}

void* customer_thread(void *arg) {
    int id = *(int*)arg;
    free(arg);

    // Seed random khusus per thread (gabungan time + id)

```

```

unsigned int seed = (unsigned int)time(NULL) ^ (id * 123456);
for (int attempt = 1; attempt <= MAX_TRIES; ++attempt) {
    // Simulasi waktu persiapan booking: random 1-3 detik
    int prep = (rand_r(&seed) % 3) + 1;
    sync_print("🔧 Customer-%d mencoba booking (percobaan %d) ... (menyiapkan
%d)s\n", id, attempt, prep);
    sleep(prep);

    // Coba ambil semaphore tanpa blokir agar bisa memberi tahu gagal segera
    if (sem_trywait(&rooms_sem) == 0) {
        // Berhasil mendapat izin kamar — sekarang cari kamar spesifik secara aman
        int room = assign_room(id);
        if (room >= 0) {
            int avail = available_rooms_count();
            sync_print("✅ Customer-%d berhasil booking! (Kamar #%d) — %d kamar
tersisa\n",
                id, room + 1, avail);

            // Menginap
            sleep(STAY_TIME);

            // Check-out: lepas kamar dan post semaphore
            release_room(room);
            sem_post(&rooms_sem);
            avail = available_rooms_count();
            sync_print("📦 Customer-%d check-out dari Kamar #%d — %d kamar
tersisa\n",
                id, room + 1, avail);
            return NULL;
        } else {
            // Ini tidak seharusnya terjadi karena semaphore menunjukkan ada slot.

```

```

        // Namun tangani defensif: kembalikan semaphore dan lanjut.
        sem_post(&rooms_sem);

        sync_print("⚠ Customer-%d: error alokasi kamar meskipun semaphore
tersedia\n", id);
    }
} else {
    // Gagal (tidak ada kamar sekarang)
    if (errno == EAGAIN) {
        sync_print("✖ Customer-%d gagal booking (kamar penuh), akan coba lagi
setelah %ds\n",
                    id, RETRY_WAIT);
    } else {
        sync_print("✖ Customer-%d gagal booking (sem_trywait error %d), coba
lagi\n", id, errno);
    }
    sleep(RETRY_WAIT);
}
}

// Setelah MAX_TRIES gagal:
sync_print("😞 Customer-%d menyerah setelah %d percobaan — tidak berhasil
booking\n", id, MAX_TRIES);
return NULL;
}

```

```

int main() {
    // Inisialisasi
    for (int i = 0; i < TOTAL_ROOMS; ++i) rooms[i] = 0;
    pthread_mutex_init(&rooms_mutex, NULL);
    pthread_mutex_init(&print_mutex, NULL);
    sem_init(&rooms_sem, 0, TOTAL_ROOMS);
}

```

```

sync_print("=== SISTEM BOOKING HOTEL ===\n");
sync_print("Total Kamar: %d\n\n", TOTAL_ROOMS);

pthread_t threads[TOTAL_CUSTOMERS];

// Buat thread customer
for (int i = 0; i < TOTAL_CUSTOMERS; ++i) {
    int *cid = malloc(sizeof(int));
    *cid = i + 1;
    if (pthread_create(&threads[i], NULL, customer_thread, cid) != 0) {
        sync_print("Error: gagal membuat thread Customer-%d\n", *cid);
        free(cid);
    }
    usleep(100000); // 0.1s jeda antar pembuatan thread agar output lebih variatif
}

// Tunggu semua selesai
for (int i = 0; i < TOTAL_CUSTOMERS; ++i) {
    pthread_join(threads[i], NULL);
}

sync_print("\nSemua customer selesai.\n");

// Bersihkan
sem_destroy(&rooms_sem);
pthread_mutex_destroy(&rooms_mutex);
pthread_mutex_destroy(&print_mutex);

return 0;
}

```

2. Output

```
qowi@QOWIAIZANI:~$ ./producer_consumer
=== SIMULASI PRODUCER-CONSUMER (Modifikasi) ===
Jumlah producer: 2 | Jumlah consumer: 2 | Buffer size: 3

👤 Producer-1 membuat item 100 (slot 0)
👤 Producer-2 membuat item 200 (slot 1)
👤 Consumer-1 mengambil item 100 (slot 0)
👤 Consumer-2 mengambil item 200 (slot 1)
👤 Producer-1 membuat item 101 (slot 2)
👤 Producer-2 membuat item 201 (slot 0)
👤 Consumer-2 mengambil item 101 (slot 2)
👤 Producer-2 membuat item 202 (slot 1)
👤 Producer-1 membuat item 102 (slot 2)
👤 Consumer-1 mengambil item 201 (slot 0)
👤 Producer-1 membuat item 103 (slot 0)
👤 Consumer-2 mengambil item 202 (slot 1)
👤 Producer-2 membuat item 203 (slot 1)
👤 Consumer-1 mengambil item 102 (slot 2)
👤 Producer-1 membuat item 104 (slot 2)
👤 Consumer-2 mengambil item 103 (slot 0)
👤 Consumer-1 mengambil item 203 (slot 1)
👤 Producer-2 membuat item 204 (slot 0)
👤 Producer-1 membuat item 105 (slot 1)
👤 Consumer-2 mengambil item 104 (slot 2)
👤 Producer-2 membuat item 205 (slot 2)
👤 Consumer-1 mengambil item 204 (slot 0)
👤 Producer-1 membuat item 106 (slot 0)
👤 Consumer-2 mengambil item 105 (slot 1)
👤 Producer-2 membuat item 206 (slot 1)
👤 Consumer-1 mengambil item 205 (slot 2)
👤 Producer-1 membuat item 107 (slot 2)
👤 Consumer-2 mengambil item 106 (slot 0)
👤 Producer-2 membuat item 207 (slot 0)
👤 Consumer-1 mengambil item 206 (slot 1)
👤 Producer-1 membuat item 108 (slot 1)
👤 Consumer-2 mengambil item 107 (slot 2)
👤 Consumer-1 mengambil item 207 (slot 0)
👤 Producer-2 membuat item 208 (slot 2)
👤 Producer-1 membuat item 109 (slot 0)
👤 Consumer-2 mengambil item 108 (slot 1)
👤 Consumer-1 mengambil item 208 (slot 2)
👤 Producer-2 membuat item 209 (slot 1)
👤 Consumer-2 mengambil item 109 (slot 0)
👤 Consumer-1 mengambil item 209 (slot 1)

✅ Semua proses selesai tanpa error!
qowi@QOWIAIZANI:~$
```


Penjelasan singkat bagaimana program bekerja

- `sem_t rooms_sem` diinisialisasi `TOTAL_ROOMS (5)`. Semaphore ini memastikan tidak lebih dari 5 customer mendapat izin mengisi kamar sekaligus.
- Ketika customer mencoba booking, thread melakukan `sem_trywait()`:
 - Jika berhasil -> berarti ada slot kamar yang tersedia; thread lalu mengunci `rooms_mutex` untuk memilih kamar fisik pertama yang kosong (`assign_room`), print, dan menginap (`sleep 5s`). Setelah check-out, membebaskan slot kamar (`release_room`) dan `sem_post()` untuk menambah ketersediaan.
 - Jika gagal (`sem_trywait` mengembalikan `EAGAIN`) -> berarti kamar penuh sekarang; thread menunggu `RETRY_WAIT` detik lalu mencoba lagi (maks 3 kali).
- `print_mutex` dipakai untuk melindungi semua operasi `printf` agar output tetap rapi dan tidak saling bertumpuk.
- `rooms_mutex` melindungi akses ke array `rooms[]` sehingga tidak ada dua thread yang meng-assign kamar yang sama (race pada struktur data).

3. Bagaimana race condition

- Race condition pada jumlah kamar dicegah oleh semaphore: semaphore menjaga jumlah thread yang boleh “memegang” kamar secara bersamaan tidak melebihi kapasitas (`TOTAL_ROOMS`). Jadi tidak mungkin lebih dari 5 thread masuk ke jalur sukses booking.
- Race condition pada alokasi kamar fisik (array `rooms[]`) ditangani oleh mutex `rooms_mutex` saat mencari dan menandai kamar yang kosong. Ini memastikan dua thread tidak menandai kamar yang sama.
- Output acak / tercampur dicegah oleh mutex `print_mutex`, sehingga setiap baris log dicetak atomically.

4. Analisis: Apa yang terjadi jika tidak menggunakan sinkronisasi?

- Jika tidak ada semaphore: banyak thread (lebih dari 5) bisa "menganggap" kamar tersedia dan akan mengisi array `rooms[]` tanpa batas → bisa menyebabkan

double-booking (dua customer bertempat di kamar sama) dan inkonsistensi pada status kamar.

- Jika tidak ada mutex untuk `rooms[]`: dua thread dapat membaca bahwa kamar X kosong lalu keduanya men-set `rooms[X]=1` → double-booking; timing race menghasilkan perilaku nondeterministik.
- Jika tidak ada mutex untuk print: output `printf` dari beberapa thread bisa saling bercampur di terminal sehingga tidak terbaca/rusak.
- Kesimpulan: tanpa sinkronisasi, program akan mengalami race condition, data korup, double-assignment kamar, dan output yang kacau.